Container communication in Docker

In this train, we'll learn how containers communicate with each other and with the outside world, focusing on container linking and network port mapping. We'll delve into how to configure and manage container communications effectively.

Learning objective

- Understand how containers communicate with each other and with the outside world, including container linking and network port mapping.

Introduction to Container Communication

Docker containers are isolated environments that run applications. However, in real-world scenarios, containers often need to communicate with each other or with external systems. Docker provides several mechanisms to enable this communication:

- **Container Linking:** A legacy method for connecting containers.
- **Network Port Mapping:** Exposing container ports to the host or external networks.
- **Docker Networks:** A more modern and flexible approach to container communication.

1. Container Linking

Container linking is a legacy feature (older method) in Docker that allows containers to communicate with each other by sharing environment variables and network information.

- When you link two containers, Docker automatically sets up network connectivity between them.
- The source container (the one being linked) exposes its environment variables and network details to the target container.

Steps to linking containers

1. Start the source container:
   `docker run -d --name source_container my_image`
2. Link the target container to the source container:
   `docker run -d --name target_container --link source_container:alias my_im`

`age`

- `source_container` : The name of the source container.
- `alias` : An alias for the source container that the target container will use to reference it.

Limitations of Linking:

- Linking is deprecated in favor of Docker networks.
- It only works in the default bridge network.
- Limited flexibility and scalability.

2. Network Port Mapping

Port mapping allows containers to communicate with the outside world by exposing container ports to the host machine or external networks.

- Docker maps a port on the host machine to a port inside the container.
- External systems can access the containerised application via the host's IP address and the mapped port.

Steps to map ports

1. Run a Container with Port Mapping:
   `docker run -d -p host_port:container_port --name my_container my_image`
   - `host_port` : The port on the host machine.
   - `container_port` : The port inside the container..
2. Access the application:
   Use the host's IP address and the `host_port` to access the application running in the container.

3. Docker Networks

While container linking and port mapping are useful, Docker networks provide a more robust and flexible way to manage container communication.

Creating a Bridge Network

The default network for Docker containers is `bridge`, which allows communication between containers on the same host.

1. Create a custom network:
   `docker network create my_network`
2. Run two containers within the same network:
   `docker run -d --name db_container --network my_network mysql`

```
docker run -d --name web_container --network my_network nginx
```

3. Verify connectivity:
```
docker exec -it web_container ping db_container
```

Benefits of Custom Networks:

- Containers can communicate by name (DNS resolution).
- Enhanced security through network isolation.
- Easier to manage and scale applications.

Best Practices for Container Communication

1. Use Docker Networks Instead of Linking:
    - Docker networks are more flexible and scalable than container linking.
    - They support multi-host communication and advanced networking features.
2. Limit Port Exposure:
    - Only expose the necessary ports to the outside world to minimize security risks.
    - Use internal networks for inter-container communication.
3. Use Environment Variables for Configuration:
    - Pass configuration details (e.g., database URLs) to containers using environment variables.
4. Monitor Network Traffic:
    - Use tools like `docker stats` or third-party monitoring solutions to monitor container network traffic.

Conclusion

Docker provides powerful networking capabilities that enable seamless communication between containers and the external world. While **legacy container linking** is outdated, **Docker networks** offer a flexible and scalable approach for inter-container communication. Using custom networks, containers can communicate securely by name, while port mapping allows services to be accessed externally.

Key Takeaways ✖

- Containers can communicate via Docker networks rather than legacy linking.
- The **bridge network** is the default, but custom networks provide better control.
- **Port mapping (-p flag)** allows external access to containerised applications.